

INSTITUTO POLITÉCNICO DE VIANA DO CASTELO
Escola Superior de Tecnologia e Gestão

Engenharia de Software
Engenharia da Computação Gráfica e Multimédia

Gerador de Ideias com IA

Vibe Coding e CI/CD — 4.^a Atividade

Grupo:

André Freitas — 33782

Afonso Neiva — 33104

Diogo Viana — 34054

Docente: Vasco Miranda

Viana do Castelo, junho de 2026

Índice

1. Introdução	3
2. Tema e Descrição da Solução	4
2.1 Valor Gerado	4
2.2 Arquitetura do Processo de Desenvolvimento	4
3. Plataforma Escolhida e Limitações	5
3.1. Ferramentas Utilizadas	5
3.2. Exemplos de Prompts Usados	5
3.3. Limitações Encontradas	5
4. GitHub, Agile e CI/CD	7
4.1. GitHub — Versionamento e Colaboração	7
4.2. Agile/Scrum — Gestão do Projeto	7
4.3. CI/CD e Deploy Automático	7
5. Prints e Evidências	8
5.1 Aplicação Web	8
5.2 GitHub Copilot a Gerar Código	9
5.3 Repositório GitHub	9
5.4 Pull Requests e Code Review	10
5.5 Scrum Board — GitHub Projects	11
5.6 Deploy no Vercel	11
5.7 Links do Projeto	11
6. Conclusão e Análise Crítica	12
6.1 Processo e Metodologias de Desenvolvimento	12
6.2 Potencial das Ferramentas de IA	12
6.3 Limitações Encontradas	12
6.3.1 Resolução Implementada: Backend Seguro com Vercel	12
6.4 Vantagens do Trabalho Colaborativo	13
6.5 Impacto do CI/CD no Desenvolvimento	13
7. Bibliografia	14

1. Introdução

O presente relatório documenta o desenvolvimento do projeto 'Gerador de Ideias com IA', realizado no âmbito da unidade curricular de Engenharia de Software, do curso de Engenharia da Computação Gráfica e Multimédia do IPVC/ESTG, como resposta à 4.ª Atividade — Vibe Coding e CI/CD.

O objetivo desta atividade é demonstrar a aplicação de processos modernos de Engenharia de Software no desenvolvimento colaborativo de soluções digitais, integrando ferramentas de Inteligência Artificial na prototipagem e no apoio ao desenvolvimento, juntamente com práticas de versionamento, Agile/Scrum, CI/CD e deploy automatizado.

Para a concretização deste trabalho, o grupo utilizou o GitHub Copilote o Codex para geração de código assistida por IA, o GitHub para versionamento e colaboração, o GitHub Projects para gestão Agile/Scrum, e o Vercel para deploy automático com integração CI/CD.

2. Tema e Descrição da Solução

A solução desenvolvida consiste numa aplicação web denominada 'Ideias IA'. O utilizador escreve um tema ou área de interesse numa caixa de texto e, ao clicar no botão 'Gerar Ideias', a aplicação recorre a um modelo de linguagem (LLM) via API para gerar cinco ideias criativas e práticas relacionadas com o tema introduzido.

Numa fase posterior, foi adicionada uma funcionalidade extra: cada ideia gerada é apresentada como um botão clicável que, quando selecionado, realiza uma nova chamada à API para aprofundar essa ideia específica, apresentando passos práticos, recursos necessários e potenciais benefícios.

2.1. Valor Gerado

A aplicação tem utilidade prática para qualquer pessoa que necessite de inspiração criativa ou precise de explorar novas ideias de forma rápida. É uma ferramenta simples, acessível e intuitiva que demonstra o potencial das APIs de IA no contexto de aplicações web.

2.2. Arquitetura do Processo de Desenvolvimento

A aplicação foi desenvolvida com tecnologias web base: HTML5, CSS3 e JavaScript, organizados em três ficheiros separados (index.html, style.css e script.js). A lógica de interação do utilizador encontra-se no script.js, que envia o pedido à rota interna /api/ideias e renderiza dinamicamente os resultados no DOM; a comunicação efetiva com a API da OpenAI passou a ser feita pelo backend serverless (ideias.js), conforme descrito na secção 6.3.1.

3. Plataforma Escolhida e Limitações

3.1. Ferramentas Utilizadas

- GitHub Copilot (VS Code) / Codex — geração de código assistida por IA
- Visual Studio Code — editor de código
- GitHub — versionamento e colaboração
- GitHub Projects — gestão Agile/Scrum
- Vercel — deploy automático e CI/CD
- OpenAI / Gemini API — geração de ideias com IA

Vantagens

- O GitHub Copilot e o Codex aceleraram significativamente o desenvolvimento, gerando código funcional a partir de descrições em linguagem natural.
- O Vercel integra-se nativamente com o GitHub, automatizando o deploy a cada push para a branch main sem qualquer configuração adicional.
- O GitHub Projects permite gerir o backlog e os sprints diretamente no repositório, facilitando a rastreabilidade entre issues e código.

3.2. Exemplos de Prompts Usados

Durante o desenvolvimento, foram utilizados os seguintes prompts no GitHub Copilot:

1. *"Cria uma aplicação web completa numa única página (HTML, CSS e JavaScript num só ficheiro index.html) chamada 'Gerador de Ideias com IA'. O utilizador escreve um tema numa caixa de texto, clica num botão 'Gerar Ideias', e a app usa a API de IA para gerar 5 ideias criativas relacionadas com esse tema. Design moderno, limpo e responsivo."*
2. *"Separa o meu ficheiro index.html em 3 ficheiros distintos: index.html apenas com a estrutura HTML, style.css com todo o CSS, e script.js com todo o JavaScript e a chamada à API. Garante que o index.html faz referência correta aos outros dois ficheiros."*
3. *"Melhora o style.css da aplicação para ter um design mais moderno e apelante. Adiciona: cores mais vibrantes, animações suaves nas ideias geradas, hover effects nos cartões, melhor tipografia e que seja totalmente responsivo para mobile."*
4. *"No meu projeto 'Gerador de Ideias com IA', quero que cada ideia seja um botão clicável e quando o utilizador clica, a app faz uma nova chamada à API para aprofundar essa ideia específica e apresenta conteúdo mais detalhado — passos práticos, recursos necessários e benefícios."*

3.3. Limitações Encontradas

- Segurança da API Key: O GitHub bloqueou automaticamente o push ao detetar a chave da API no código, exigindo autorização manual. Em produção, a solução correta seria utilizar variáveis de ambiente no servidor.
- Limitações da API gratuita: A Gemini API apresentou erros de 'high demand' em períodos de maior utilização, exigindo a mudança para o modelo gemini-1.5-flash. Obrigando assim a mudança para a API da OPENAI.

- CI/CD simplificado: O fluxo de CI/CD implementado é básico, sem testes automatizados no pipeline. Uma implementação mais completa incluiria GitHub Actions com execução de testes antes do deploy.

4. GitHub, Agile e CI/CD

4.1. GitHub — Versionamento e Colaboração

O repositório GitHub foi utilizado como base de todo o desenvolvimento. Cada funcionalidade foi desenvolvida numa branch dedicada, seguindo a convenção feature/nome-da-funcionalidade, garantindo um histórico de desenvolvimento claro e organizado.

As branches criadas foram:

- feature/setup-inicial — estrutura base HTML/CSS/JS
- feature/api-ia — integração da API de IA
- feature/logica-ideias — melhoria da lógica e UX
- feature/design — melhorias visuais e responsividade
- feature/testes — testes unitários e correção de bugs
- feature/nova-funcionalidade — ideias clicáveis com aprofundamento
- origin/design-melhoria — mais melhorias de design
- origin/api/problem — mudança no código para corrigir problema da api ia
- origin/design-v02 — versão final do design da app

Cada branch originou um Pull Request que foi revisto e aprovado por um elemento diferente do grupo antes do merge para a main, garantindo a prática de code review colaborativo.

4.2. Agile/Scrum — Gestão do Projeto

Foi utilizado o GitHub Projects como plataforma de gestão Agile/Scrum, com um board Kanban organizado nas colunas To Do, In Progress e Done.

O Sprint concluído incluiu os seguintes itens do backlog:

1. Setup inicial HTML/CSS/JS
2. Configurar Vercel e deploy automático
3. Integrar API de IA
4. Lógica de gerar ideias
5. Design e estilo da página
6. Testes e correção de bugs

As tarefas foram distribuídas pelos três elementos do grupo, com cada issue atribuído a um responsável. A gestão de bugs foi feita através da criação de issues específicos no repositório.

4.3. CI/CD e Deploy Automático

O deploy automático foi configurado através do Vercel, uma plataforma de hosting que se integra nativamente com o GitHub. A cada merge para a branch main, o Vercel deteta automaticamente as alterações e executa um novo deploy, disponibilizando a versão mais recente da aplicação online em segundos.

Este fluxo implementa um pipeline básico de CI/CD: o código é integrado via Pull Requests revistos por colegas (Continuous Integration) e entregue automaticamente em produção após o merge (Continuous Deployment).

5. Prints e Evidências

Nesta secção apresentam-se as capturas de ecrã das principais componentes do projeto.

5.1. Aplicação Web

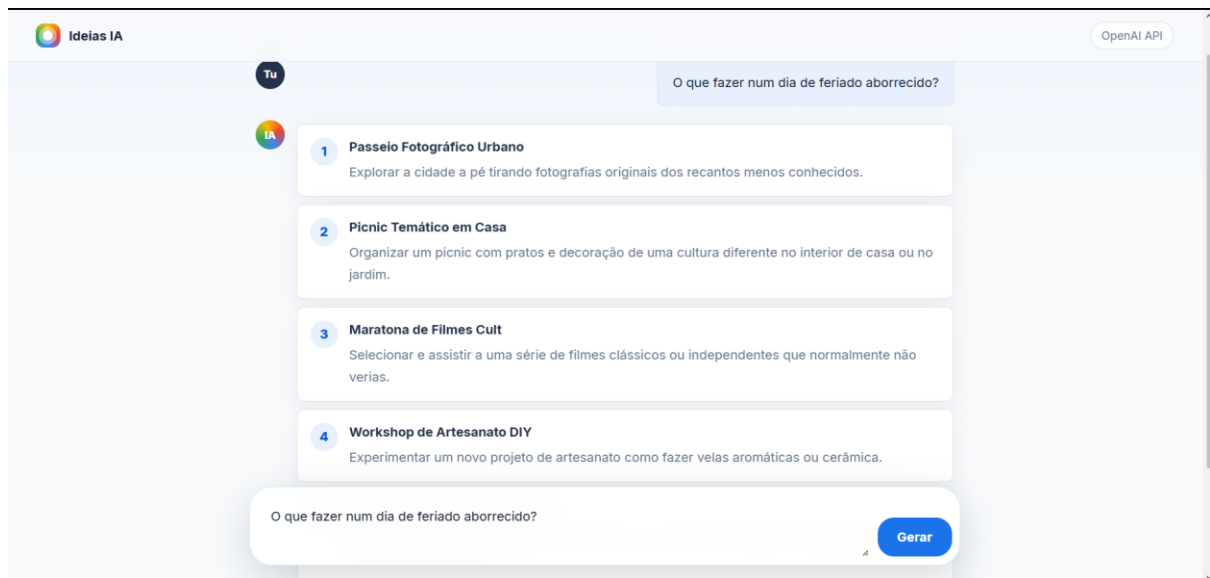


Figura 1 — Interface do Gerador de Ideias com IA

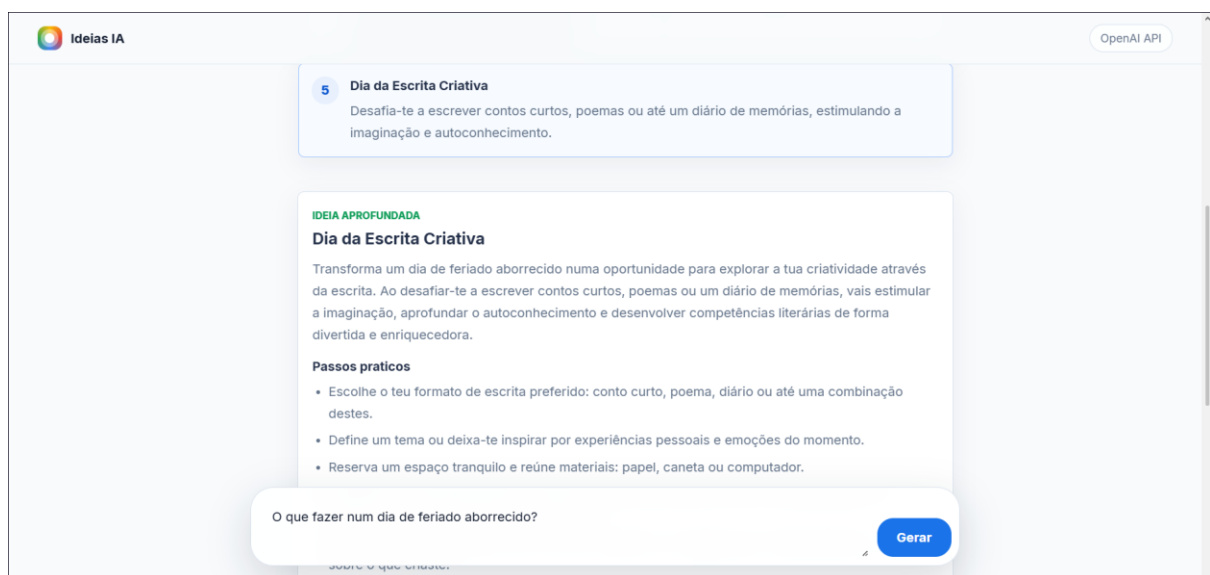


Figura 2 — Interface do Gerador de Ideias com IA, ideia aprofundada

5.2. GitHub Copilot a Gerar Código

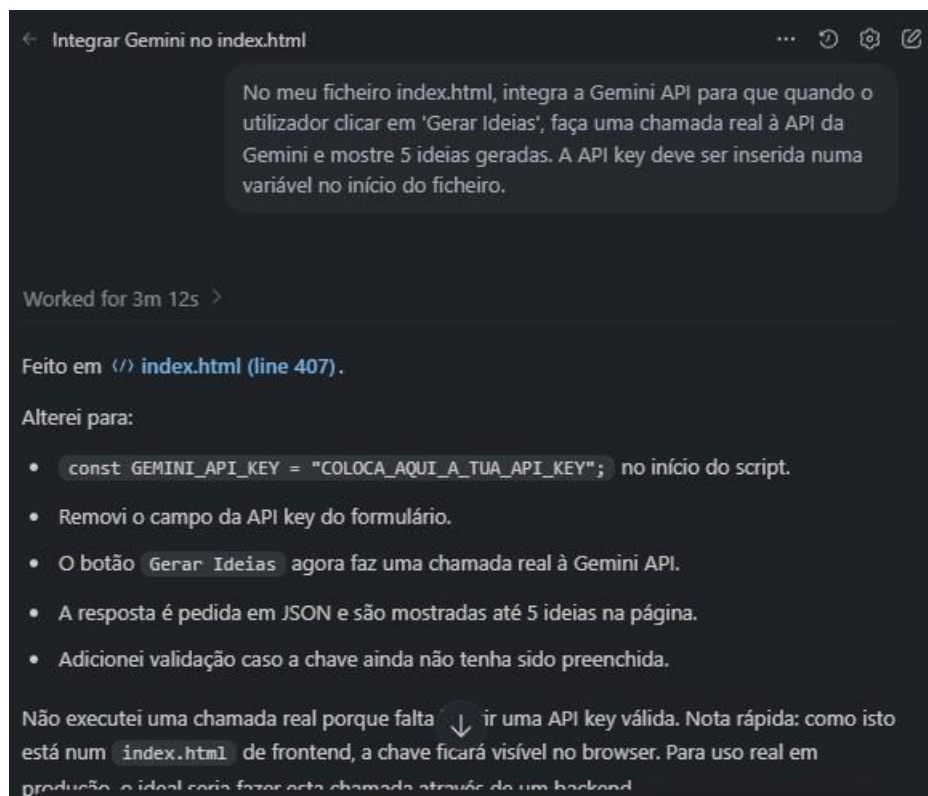


Figura 3 — GitHub Copilot a gerar código

5.3. Repositório GitHub

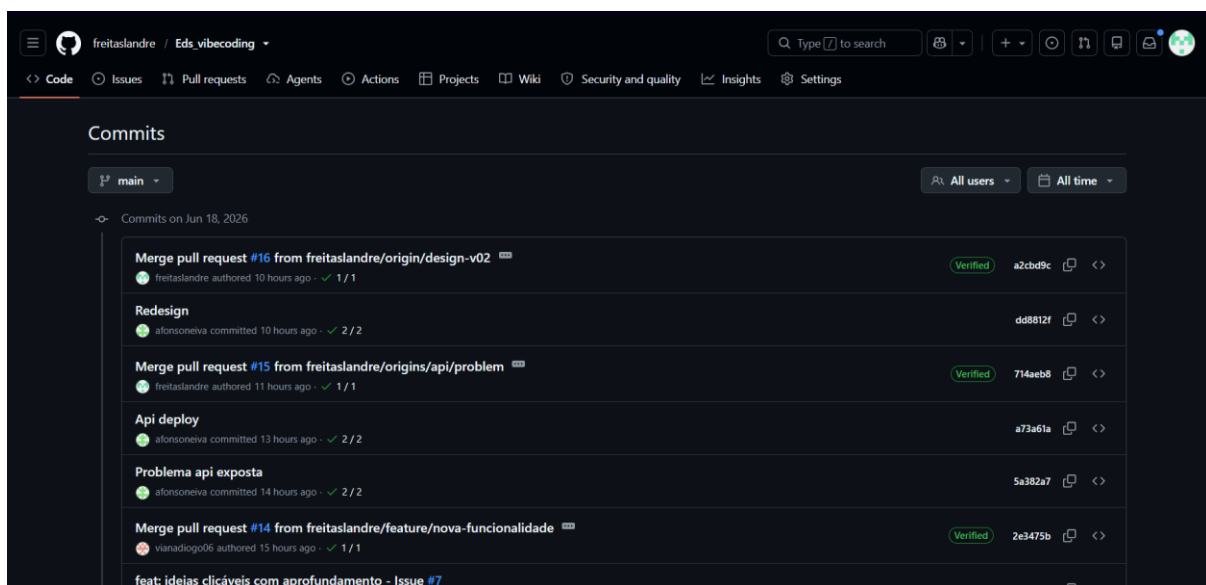


Figura 4 — Histórico de commits

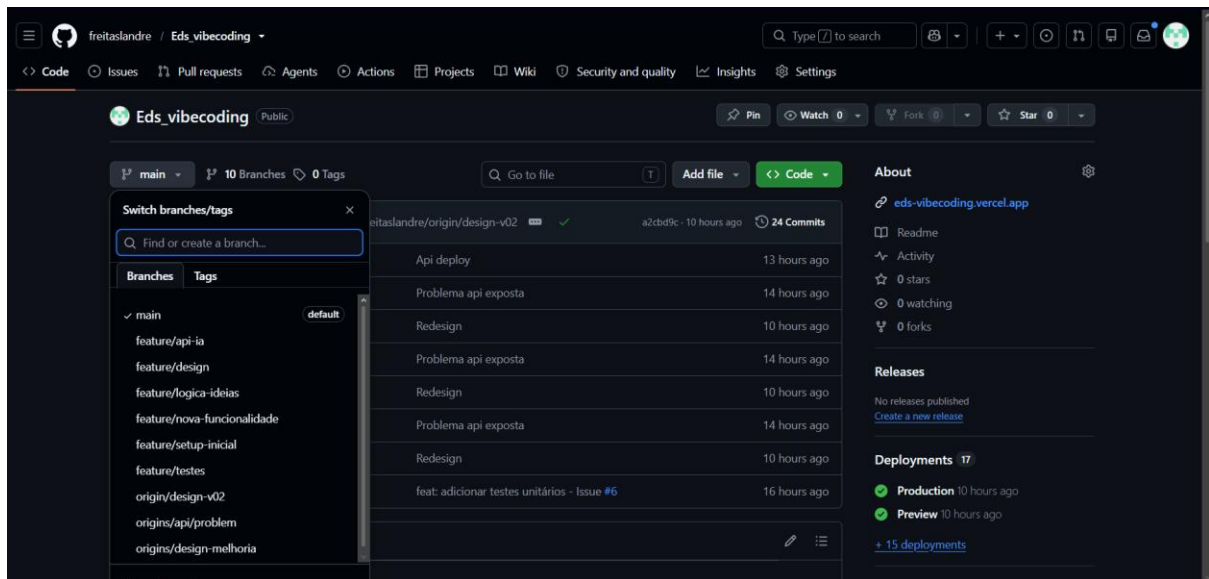


Figura 5 — Histórico de branches

5.4. Pull Requests e Code Review

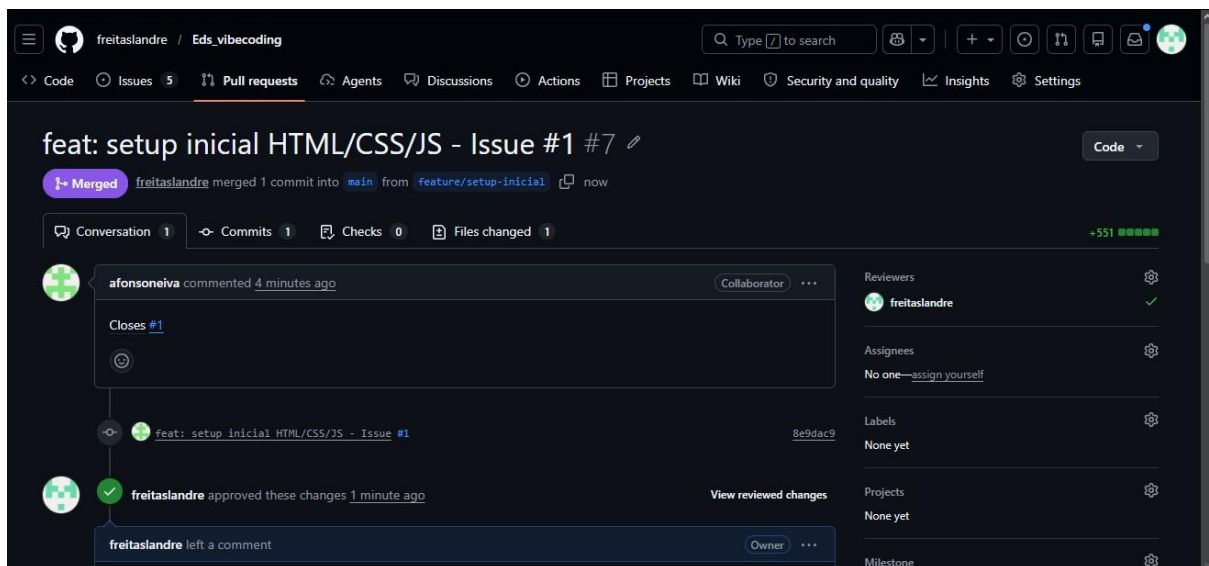


Figura 6 — Pull Request com aprovação de colega

5.5. Scrum Board — GitHub Projects

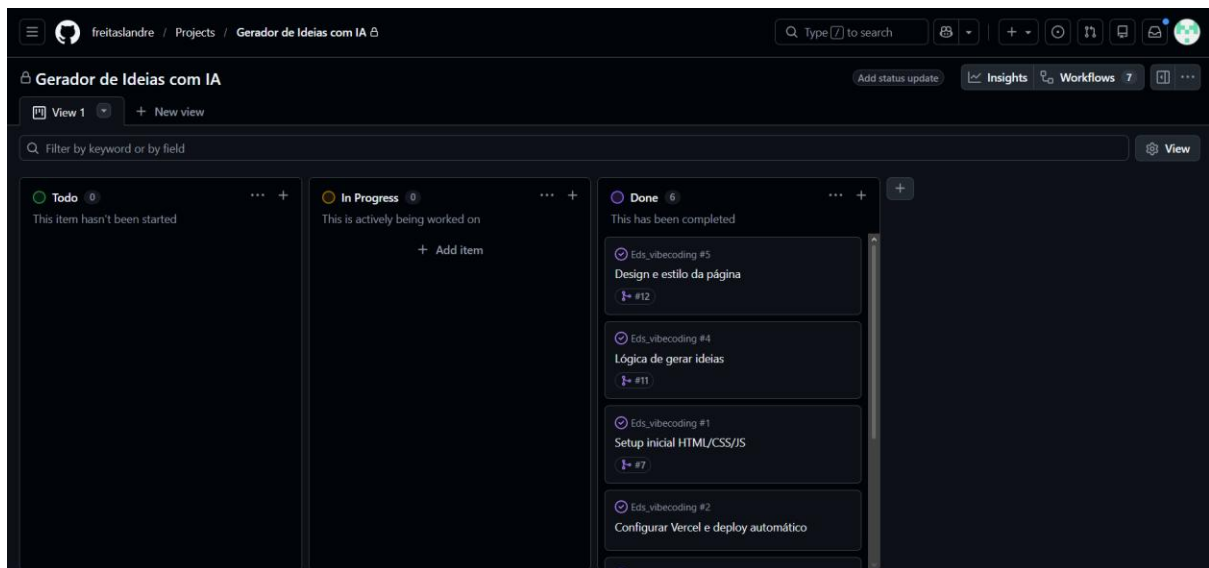


Figura 7 — Scrum Board com sprint concluída

5.6. Deploy no Vercel

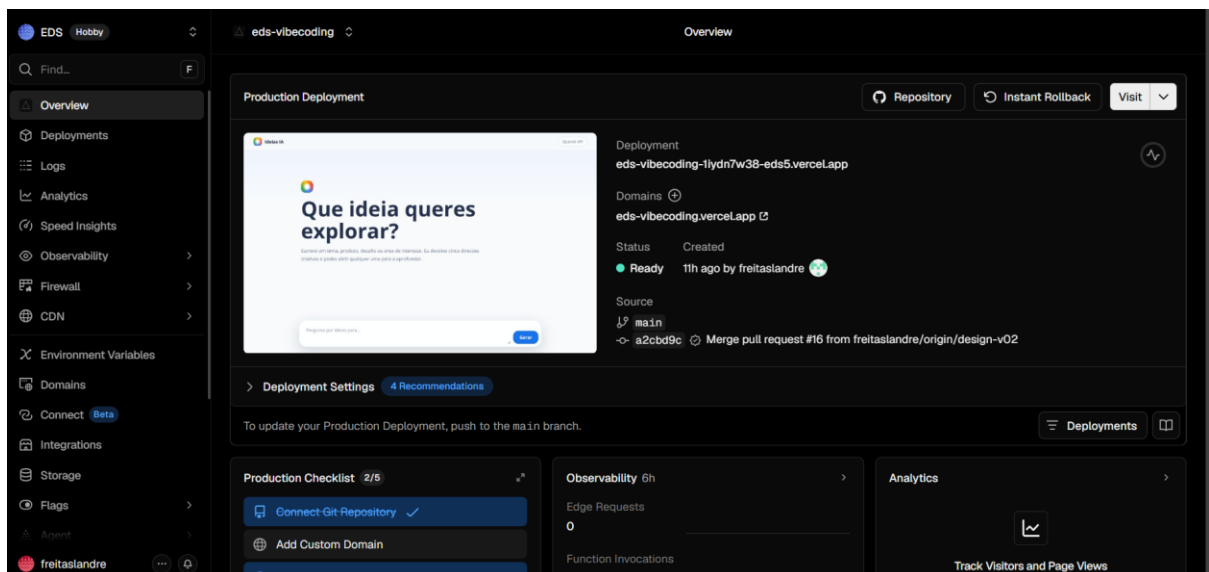


Figura 8 — Dashboard do Vercel com deploy automático

5.7. Links do Projeto

- Link da aplicação online: <https://eds-vibecoding.vercel.app>
- Link do repositório GitHub: https://github.com/freitaslandre/Eds_vibecoding.git

6. Conclusão e Análise Crítica

6.1. Processo e Metodologias de Desenvolvimento

A adoção de uma metodologia Agile/Scrum, mesmo que simplificada, revelou-se fundamental para a organização do trabalho. A divisão do projeto em issues claros e atribuídos a responsáveis específicos permitiu que os três elementos do grupo trabalhassem em paralelo de forma eficiente, sem conflitos de código significativos.

6.2. Potencial das Ferramentas de IA

O GitHub Copilot e o Codex demonstraram um potencial enorme na aceleração do desenvolvimento. Tarefas que normalmente levariam horas foram concluídas em minutos através de prompts bem formulados. No entanto, verificou-se que o código gerado nem sempre é perfeito à primeira — exige revisão, adaptação e compreensão por parte do programador. A IA é um assistente poderoso, não um substituto do pensamento crítico do desenvolvedor.

6.3. Limitações Encontradas

A principal limitação encontrada foi a gestão segura da API key. O GitHub detetou automaticamente a chave exposta no código e bloqueou o push, o que é uma funcionalidade de segurança importante mas que, neste contexto académico com HTML estático, não tem solução trivial sem a utilização de um backend. Numa aplicação real, a API key seria gerida através de variáveis de ambiente no servidor. Esta limitação foi posteriormente resolvida através da implementação de uma arquitetura de backend serverless, descrita na secção seguinte.

6.3.1. Resolução Implementada: Backend Seguro com Vercel

Para eliminar a exposição da chave da API no código do lado do cliente, o grupo implementou uma arquitetura de backend serverless através das funções API da Vercel. Após a aplicação desta solução, o erro que ocorria de forma recorrente — manifestando-se geralmente após a quarta ou quinta utilização do gerador — deixou de se verificar, passando a aplicação a funcionar de forma estável.

Passo 1 — Proteção da chave no painel da Vercel

No painel da Vercel, dentro das definições do projeto (Settings → Environment Variables), foi criada uma nova variável de ambiente com a chave `OPENAI_API_KEY`, à qual foi atribuído o valor da nova chave gerada na OpenAI. Desta forma, a chave passou a ficar armazenada de forma encriptada no servidor, nunca sendo exposta no código-fonte nem no repositório GitHub.

Passo 2 — Criação de uma rota de API interna (backend serverless)

Foi criada, na raiz do projeto, uma pasta denominada `api`, automaticamente reconhecida pela Vercel como contendo funções serverless. Dentro desta pasta foi adicionado o ficheiro `ideias.js`, responsável por comunicar de forma segura com a API da OpenAI, funcionando como intermediário entre o frontend e o serviço externo:

```
export default async function handler(req, res) {
  const tema = req.body.tema;
  const resposta = await fetch("https://api.openai.com/v1/chat/completions", {
    method: "POST",
    headers: {
      "Authorization": `Bearer ${process.env.OPENAI_API_KEY}`,
```

```

    "Content-Type": "application/json"
  },
  body: JSON.stringify({
    model: "gpt-4o-mini",
    messages: [{
      role: "user",
      content: `Gera 5 ideias criativas sobre: ${tema}`
    }]
  })
});
const dados = await resposta.json();
res.status(200).json(dados);
}

```

Passo 3 — Atualização da interface (frontend)

O frontend deixou de comunicar diretamente com a OpenAI, passando a efetuar pedidos apenas à rota interna `/api/ideias`, que oculta por completo a chave da API ao utilizador final:

```

const resposta = await fetch("/api/ideias", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ tema: "benfica" })
});
const resultado = await resposta.json();

```

Passo 4 — Deploy e validação

Após o envio das alterações para o repositório GitHub, a Vercel reconstruiu automaticamente o projeto e ativou a nova rota interna. Com esta arquitetura em produção, a aplicação passou a responder corretamente em todas as utilizações, com a chave da API totalmente protegida contra exposição acidental.

6.4. Vantagens do Trabalho Colaborativo

O trabalho colaborativo com GitHub permitiu que os três elementos trabalhassem em simultâneo em funcionalidades diferentes, sem interferências. A prática de Pull Requests com code review por outro elemento do grupo introduziu uma camada de qualidade ao desenvolvimento, permitindo detetar erros e sugerir melhorias antes da integração na branch principal.

6.5. Impacto do CI/CD no Desenvolvimento

A integração do Vercel com o GitHub transformou radicalmente a experiência de deploy. Em projetos tradicionais, o processo de publicar uma atualização é manual e demorado. Com o CI/CD configurado, bastou fazer merge para a main para ter a aplicação atualizada online em segundos. Esta automatização reduz o erro humano, aumenta a frequência de entregas e acelera o ciclo de desenvolvimento.

Em suma, esta atividade demonstrou que a combinação de ferramentas de IA, práticas de versionamento colaborativo e automação de deploy constitui um stack moderno e eficaz para o desenvolvimento de software, com aplicação direta no mercado de trabalho atual.

7. Bibliografia

Anthropic. (2024). Claude AI assistant. <https://www.anthropic.com>

GitHub. (2024). GitHub Copilot documentation. <https://docs.github.com/en/copilot>

GitHub. (2024). GitHub Projects documentation. <https://docs.github.com/en/issues/planning-and-tracking-with-projects>

Google. (2024). Gemini API documentation. <https://ai.google.dev/docs>

OpenAI. (2024). OpenAI API reference. <https://platform.openai.com/docs>

Schwaber, K., & Sutherland, J. (2020). The Scrum guide. <https://scrumguides.org>

Vercel. (2024). Vercel deployment documentation. <https://vercel.com/docs>